

Cloud Infrastructure and Services Laboratory (LPEIT-115)

MINI PROJECT FILE

GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA

BACHELOR OF TECHNOLOGY

(INFORMATION TECHNOLOGY)



SUBMITTED BY

SUBMITTED TO:

NAME: ALISHA, MEHARDEEP KAUR

Pf. KAMALJIT KAUR

BRANCH:IT A

UNIVERSITY ROLL NO:2203792, 2203856

DEPARTMENT OF INFORMATION TECHNOLOGY

GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

Iris Flower Classification Web Application

1. Introduction

The Iris Flower Classification Web Application is a lightweight and user-friendly web application developed using Flask. It allows users to predict the species of an Iris flower by entering four numerical attributes: Sepal Length, Sepal Width, Petal Length, and Petal Width. A pre-trained machine learning model classifies the flower into one of three species: Setosa, Versicolor, or Virginica.

This project demonstrates the integration of machine learning with web development and DevOps practices. It also showcases cloud deployment using **AWS services such as EC2, ECS, and IAM**, along with containerization and CI/CD automation.

2.Objective

The main objectives of this project are:

- To provide an intuitive web interface for predicting Iris flower species.
- To perform real-time machine learning inference on user input.
- To deploy the application on **AWS cloud infrastructure**.
- To integrate **AWS services** into the CI/CD and deployment workflow.

3. Technology Stack

Frontend

- HTML5 for structuring the web pages
- CSS3 for styling and improving user experience

Backend

- Python 3.x as the core programming language
- Flask for handling HTTP requests and routing
- Scikit-learn for machine learning model development and prediction

DevOps & Cloud

- Docker for containerizing the application
- Jenkins for continuous integration and continuous deployment
- AWS EC2 for hosting compute resources
- AWS ECS for managing and running Docker containers
- AWS IAM for secure authentication, authorization, and access control

4. System Architecture

1. **User Interface:** Accepts four numerical inputs related to the Iris flower.
2. **Flask Backend:** Processes user input and sends it to the trained ML model.
3. **ML Model:** A serialized Scikit-learn model used for prediction.
4. **Docker & Jenkins:** Used together for containerization and CI/CD automation.
5. **AWS Services:**

- **EC2** for hosting infrastructure
- **ECS** for container orchestration
- **IAM** for secure access and role management

Dataset

The Iris dataset from the UCI Machine Learning Repository is used. It contains 150 samples evenly distributed among three species with the following features:

- Sepal Length (cm)
- Sepal Width (cm)
- Petal Length (cm)
- Petal Width (cm)

Model Training

- Algorithm: Decision Tree Classifier
- Preprocessing: No missing values; normalization not required
- Serialization: Model saved using joblib for runtime predictions

Flask Web Application

Features

- Simple HTML form for user input
- Flask endpoint processes the request
- Prediction result displayed on the web page

Project Structure

- app.py: Main application logic
- templates/: HTML files
- static/: CSS styling

Docker Integration

Docker is used to containerize the Flask application.

- Creates a consistent runtime environment
- Simplifies deployment on AWS infrastructure

Docker Compose has been removed to keep the setup minimal.

Jenkins CI/CD Pipeline

Jenkins automates the build and deployment process.

- Pulls source code from GitHub
- Builds Docker image
- Pushes and deploys the application on **AWS infrastructure**
- Integrates AWS credentials and services within the pipeline

Deployment & Access

The application is deployed on AWS cloud services.

Access link is provided through the **AWS deployment endpoint**.

Testing

- Manual testing via the web interface
- Unit testing using pytest
- Container-level verification after deployment

Predict the Iris Flower Class

Sepal Length:

Sepal Width:

Petal Length:

Petal Width:

Predict

Predicted Iris Class: 2

Conclusion

The Iris Flower Classification Web Application successfully demonstrates the complete lifecycle of a machine learning project, starting from model training and web integration to automated deployment on cloud infrastructure. This project goes beyond basic model implementation by incorporating modern DevOps and cloud computing practices, making it a strong example of a production-ready machine learning application.

At its core, the application integrates a trained machine learning model with a Flask-based web interface, allowing users to perform real-time predictions in an intuitive and interactive manner. The simplicity of the Iris dataset makes the project easy to understand, while the overall architecture reflects real-world deployment scenarios. By containerizing the application using Docker, the project ensures consistency across development, testing, and production environments.

A key strength of this project lies in its integration with Amazon Web Services (AWS). The use of AWS EC2 provides reliable and scalable compute resources, enabling the application to run efficiently in a cloud environment. AWS ECS plays a critical role in managing and orchestrating Docker containers, simplifying deployment and ensuring that the application can be scaled and managed effectively. Additionally, AWS IAM enhances the security of the system by implementing role-based access control, ensuring that AWS resources are accessed securely and according to best practices.

The implementation of a Jenkins-based CI/CD pipeline further strengthens the project by automating the build, testing, and deployment processes. Through Jenkins, the application automatically pulls code from GitHub, builds Docker images, and deploys them to AWS infrastructure. This automation minimizes manual intervention, reduces deployment errors, and demonstrates how continuous integration and continuous deployment workflows are used in modern cloud-based applications.

Future Scope

- The application can be extended to support **additional flower or plant species datasets**, enabling multi-class or multi-dataset classification.
- More advanced machine learning models such as **Random Forest, Support Vector Machines, or Deep Learning models** can be implemented to improve prediction accuracy.
- Integration of **AWS Auto Scaling** can enhance application scalability based on real-time traffic demands.
- The system can be enhanced by adding **AWS Load Balancer** to improve availability and fault tolerance.
- Real-time monitoring and logging can be implemented using **AWS CloudWatch** for better performance tracking and debugging.
- The application can be upgraded to a **fully serverless architecture** using AWS Lambda and API Gateway.

References (IEEE Format)

- [1] UCI Machine Learning Repository, “Iris Data Set,” [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/iris>
- [2] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [3] Docker Inc., “Docker Documentation,” [Online]. Available: <https://docs.docker.com/>
- [4] Jenkins, “Jenkins User Documentation,” [Online]. Available: <https://www.jenkins.io/doc/>
- [5] Amazon Web Services, “Amazon EC2 User Guide,” [Online]. Available: <https://docs.aws.amazon.com/ec2/>
- [6] Amazon Web Services, “Amazon ECS Developer Guide,” [Online]. Available: <https://docs.aws.amazon.com/ecs/>